

ARISTOTLE UNIVERSITY OF THESSALONIKI

ARISTURTLE

ELECTRIC OVEN

Project Subtitle

BY KOUIDIS DIMITIRIS

Table of Contents

Table of Contents 2
Introduction 3
1. HARDWARE 4
1.1 Zero-Volt Cross Detection Circuitry 4
1.2 Phase Control 6
1.3 Temperature Reading 7
1.4 Hall sensor 7
2. Peripherals 8
2.1 LCD Display 8
2.2 Thermocouples 8
2.3 Potentiometer 8
2.4 Digital Temperature Sensors 8
2.5 Safety Digital Temperature Sensor 9
3. Control System 10
3.1 Brief to PI control 10
3.2 Esp32 11
4.Temporary implementation 12
5. Test and suggestions 22
Sources – References 23

Introduction

This year we decided to re-manufacture the external electric oven that operates with a PI control in combination with a phase control system. This decision was made after noticing that the previous system could not keep the temperature close to the desired one and the declinations during the function of the oven were large, while the mechanical team was not able to use the smaller oven located on the *Oikiskos B* for everything they needed.

To provide a solution to this problem, we reconstructed the oven from scratch, we designed a new PCB using a better version of the Zero Cross Detection circuitry and added some extra safety features.

This booklet's purpose is to explain the basic functions of the electric oven, providing explanations of the circuits used in the PCB, the software used and the improvements that can be made.

By the time this booklet is being written, there is no software implementation for the actual PCB but software for a "version" of it as the time was limited and we implement a faster, maybe not so reliable, solution. The booklet contains both the current implementation and the "correct" one.

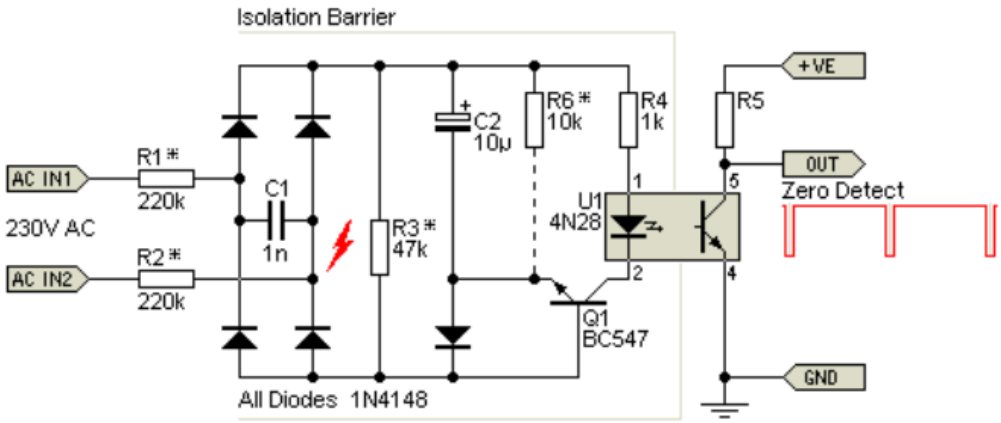
1. HARDWARE

1.1 Zero-Volt Cross Detection Circuitry

As shown on the image below, we want to detect the times that the alternating voltage passes the 0V point and each time it does send a signal/pulse to our microcontroller. With blue we represent the alternating voltage and with red the pulses.



To achieve this, we needed to find a circuit that does this exact thing, while degrading the voltage from 220V to a value much smaller.

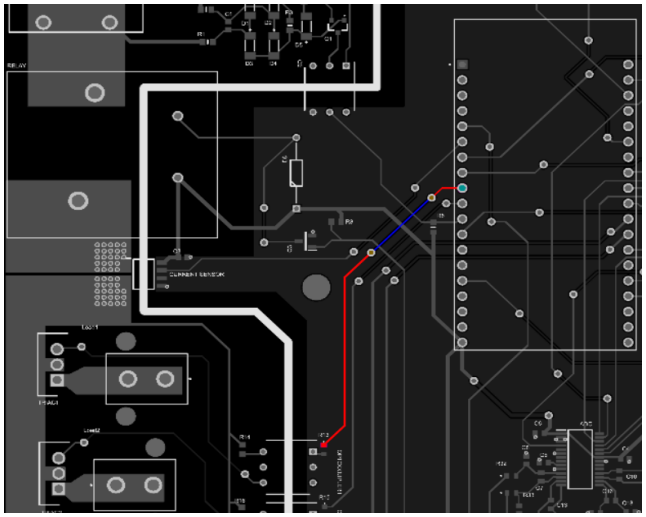


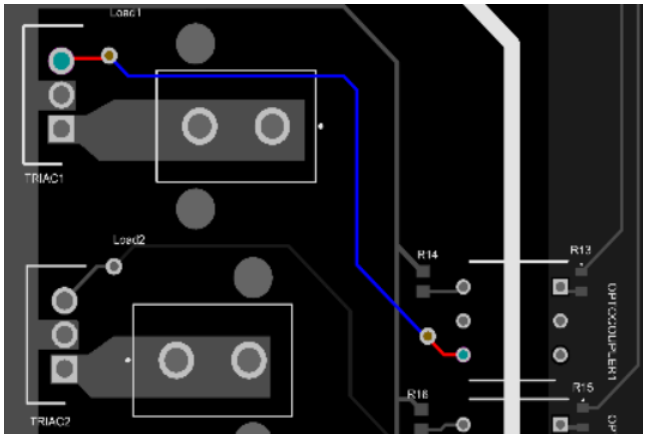
The above circuit was found online and tested before deciding whether to use it or not. During the tests we noticed that the results were satisfactory and better than the previous implementation for the detections of the zero cross.

You can find the analytical description of the circuitry in the Sources-References chapter on links [5] and [6].

1.2 Phase Control

After the zero-volt cross detection the pi controller, implemented with the esp32, decides when the firing pulse will be sent. To achieve the desired load, we use four optocouplers, so we need four firing pulses. The firing pulses are being sent by some pins of the esp32. Each of these pins are connected to an optocoupler. We need the optocoupler for isolation between the esp32 and the triacs. The outlet of each optocoupler is connected with the trigger gate of one triac and in that way the triac receives the triggering pulse of the esp32, and it conducts.





1.3 Temperature Reading

In addition to the improvement of the zero-volt cross detection circuit, we decided to improve the measurement of the oven's internal temperature by using four thermocouples. To read their values we use an ADC compatible with the SPI protocol. Each thermocouple is placed on one of the edges of the oven and with the ADC we alternate the readings between them every few milliseconds. By that way we can calculate the average temperature of the four thermocouples and have a clear view of the setup's function

1.4 Hall sensor

For safety reasons we decided to add a Hall sensor to measure the current flowing through the triacs. The sensor sends the measurement to esp32 and can be printed on the Serial Monitor or shown on the LCD Display, according to your preferences.

2. Peripherals

2.1 LCD Display

As a display we used a 16×2 LCD display that interfaces with I2C protocol. On the LCD we display the current temperature and the target. The LCD holds four pins, one for 5V, one for ground (GND) and two for communication. These are the SDA and SCL pins which need to be connected to 21 and 22 pins of the esp32 respectively.

2.2 Thermocouples

A thermocouple is an electrical device consisting of two dissimilar electrical conductors forming an electrical junction. A thermocouple produces a temperature dependent voltage as a result of the Seebeck effect, and this voltage can be interpreted to measure temperature. We use four thermocouples each connected to the ADC.

2.3 Potentiometer

The potentiometer is used to set the desired temperature for the oven to avoid having to change the value in the code and upload it every time a change is needed.

2.4 Digital Temperature Sensors

Apart from the thermocouples the setup supports up to four digital temperature sensors in order to verify the measurements that we get from the thermocouples. Each sensor has its own mini-PCB (which contains also a capacitor) to facilitate the connection with the main PCB. The sensors support the SPI protocol so we can alternate the readings between them and the ADC.

Notice that there is a code implementation with a digital sensor, the code of which you can find in the team's Gitlab on the unity "Little Oven".

2.5 Safety Digital Temperature Sensor

Another programmable temperature sensor is being used for safety reasons. If the temperature of the oven exceeds a certain large value (in case of malfunction) it triggers a Normally Open relay and it stops the power supply of the triacs. The sensor has its own mini-PCB (which includes three resistors and a capacitor) as well.

3. Control System

3.1 Brief to PI control

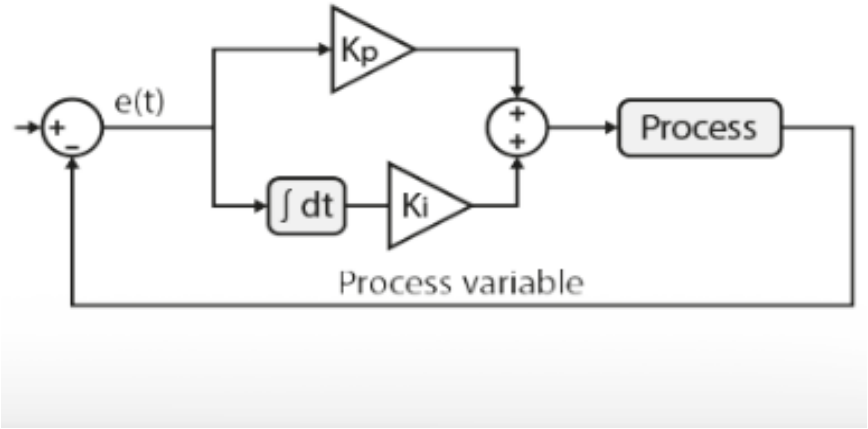
A proportional–integral controller (PI controller) is a control loop mechanism employing feedback that is widely used in industrial control systems and a variety of other applications requiring continuously modulated control. A PI controller continuously calculates an error value e(t) as the difference between a desired setpoint and a measured process variable and applies a correction based on proportional and integral terms (denoted P and I respectively).

→ **Term P** is proportional to the error e(t). For example, if the error is large and positive, the control output will be proportionately large and positive, considering the gain factor "K". Using proportional control alone will result in an error between the setpoint and the actual process value, because it requires an error to generate the proportional response. If there is no error, there is no corrective response.

→ **Term I** accounts for past values of the error and integrates them over time to produce the I term. For example, if there is a residual error after the application of proportional control, the integral term seeks to eliminate the residual error by adding a control effect due to the historic cumulative value of the error. When the error is eliminated, the integral term will cease to grow. This will result in the proportional effect diminishing as the error decreases, but this is compensated for by the growing integral effect.

Tuning

The balance of these effects is achieved by loop tuning to produce the optimal control function. The tuning constants are shown below as "K" and must be derived for each control application, as they depend on the response characteristics of the complete loop external to the controller. These are dependent on the behavior of the measuring sensor, the final control element, any control signal delays and the process itself.



Notice that for the further improvement of the setup, a Term D (derivative) can be added to the control (PID controller).

3.2 Esp32

We used an esp32-devkitC to implement PI control and the interface with the user. The last one includes the LCD display and the potentiometer.

The basic functions of the esp32 are:

- Read the desired temperature through the potentiometer
- Read the temperature through the ADC
- Read the current value through the hall sensor
- Detect the zero-volt cross
- Execute PI control to reach the desired temperature
- Print the real, the desired temperature and the error to the display

4.Temporary implementation

As mentioned at the beginning the code for all the above functionalities is non-existent. Below is described the existing code which the team used during the 2024-2025 season.

Thermistor

The thermistor we use is an NTC thermistor with a nominal resistance of 100kΩ. Thermistors are temperature-sensing elements made of semiconductor material that has been sintered to display large changes in resistance in proportion to small changes in temperature. This resistance can be measured by using a small and measured direct current, or dc, passed through the thermistor to measure the voltage drop produced. NTC Thermistors are non-linear resistors, which alter their resistance characteristics with temperature. The resistance of NTC will decrease as the temperature increases. The way the resistance decreases is related to a constant known in the electronics industry as beta, or β. Beta is measured in °K.

Before we convert the resistance of the thermistor to the respective temperature, we need to measure its resistance. For this purpose, we must turn the esp32 into an ohmmeter. First, we connected a resistor in series with the thermistor. This resistor should have the same resistance as the nominal resistance of the thermistor for better accuracy. This addition occurred under the connectors which would be used, under normal circumstances, for the external digital temperature sensors. We also added a decoupling capacitor for noise reduction.

So, we needed to define the resistor we used in series with the thermistor, the nominal resistance of the thermistor, the reference temperature and the b coefficient of our thermistor. We also initialize some other variables for the calculation of the temperature.

```
// Thermistor constants
#define SERIES  100000 // 100 kohm resistor
const float Beta = 3892.0;
const float Temp = 298.15;

// Define ESP32 pins
int THERMISTORPINS[] = {13, 33, 32, 36};
const int NUM_SENSORS = 4;
const int POTENTIOMETER = 34;


float Vin = 3.3;
float Vout = 0;
float sensorValue = 0;
float R_therm = 0;
float steinhart = 0;
float avgTemp = 0;
movingAvg steinhartmvavg(3);
```

The calculation of the temperature is implemented inside the function *read_average_temperature()*. The first step is to read the analog value from the pins of the esp32 that the thermistors are connected to and convert it to volts. Provided that we know the voltage on the thermistor, the Vin and the resistor in series with the thermistor we can calculate their resistance. At the end we calculate the average temperature.

During operation we noticed that sometimes the values from the first 2 pins were correct while 2 were not. That happened probably from some bad connections to the wire because after a point everything worked properly. However, we implemented a “check” for the values of the last 2 pins according to the measurements of the first 2. If the decline of the values is less than a threshold they are being considered, if there is more, then they are skipped.

```
float read_average_temperature() {
    float totalTemp = 0;
    int count = 0;
    float tempReadings[NUM_SENSORS];

    // Read all sensors first
    for (int i = 0; i < NUM_SENSORS; i++) {
        int sensorValue = analogRead(THERMISTORPINS[i]);
        float Vout = Vin * (sensorValue / 4095.0); // 12-bit ADC
        float R_therm = SERIES * Vout / (Vin - Vout);
        float steinhart = log(R_therm / SERIES);
        steinhart = (1.0 / Temp) + (steinhart / Beta);
        steinhart = 1.0 / steinhart;
        float temperatureC = steinhart - 273.15;
        tempReadings[i] = temperatureC;
    }
    // Compute average of first 2
    float refAvg = (tempReadings[0] + tempReadings[1]) / 2.0;
    totalTemp = tempReadings[0] + tempReadings[1];
    count = 2;
    // Check 3rd and 4th sensors
    for (int i = 2; i < NUM_SENSORS; i++) {
        if (abs(tempReadings[i] - refAvg) <= 6.0) {
            totalTemp += tempReadings[i];
            count++;
        }
    }
    avgTemp = totalTemp / count;
    return steinhartmvavg.reading(avgTemp);
}
```

Zero-Volt Cross

At the beginning we define the pin where the output from the zero-volt cross detection circuit will be written and the pins which will “send” the pulse to the optocouplers. We also define a boolean value which turns to “1” when a zero-volt cross is detected to help us handle the interrupt we are using for this purpose.

```
const int zcd = 35; // Zero cross detection pin
const int angleControl[] = {25,26,17,16};  // Trigger pins for TRIAC
volatile bool zero_cross_detected = false;
```

Below is provided the interrupt handler. In other words when the zcd pin goes to LOW (according to our circuit zcd is always HIGH and is pulled down when a zero cross is detected) and the error is bigger than zero then we allow the pulses from the angleControl pins to be sent to the optocouplers so that the triacs will conduct. We will explain this in more detail below.

```
void IRAM_ATTR zero_cross_detection() {
    zero_cross_detected = true;
}
```

PI implementation

Continuing from the ZCD, let's see how we handle PI control. Firstly, we declare the values of the variables used to calculate the PI output at each state. The PI output (variable "pi") is the value which determines for how long the angleControl pins will be set as HIGH --> how many amperes will conduct through the triacs. To calculate its value, we use the formula pi = error * Kp + previous_error * Ki as can be seen below.

After manual testing, we found that Kp = 200 and Ki = 20 will give the best results for our implementation.

```
// PI Variables
int error = 0;
int previous_error = 0;
int totalError = 0;

float kp = 200;
float ki = 20;

float proportional = 0;
float integral = 0;
float pi = 0;
```

To not burn the triacs, we must set constraints on the pi variable. In the PCB placed on *Oikiskos B* at this time, there were problems with two of the four triacs and they didn't conduct. After hours of debugging, we could not find what was wrong. So, we supplied the four resistors from the two triacs. Having a maximum rating of 10A, after manual testing, we found out that the upper limit of pi must be 2600. Under normal circumstances and if all triacs are working properly this upper limit can be exceeded, while there is no limitation for the lower boundary. We strongly recommend that before setting the upper limit to manually test with an amperemeter the load and slightly increase it.

```
// PI Control
void pid() {

    previous_error = error;
    error = target - round(avgTemp); //or -steinhart

    if (manualPI) return;

    if (previous_error * error < 0) {
      totalError = 0;
    } else {
      totalError += error;
      totalError = constrain(totalError, -200, 200);
    }

    proportional = error * kp;
    integral = totalError * ki;
    pi = proportional + integral;

    pi = constrain(pi, 330, 2600);
}
```

To gradually increase the upper boundary, we have implemented a feature where you can change manually the PI output by writing on the Serial Monitor “PI= “and the value you want to set.

To switch back to the automatic PI control just type “AUTO”.

```
if (Serial.available()) {
String input = Serial.readStringUntil('\n');
input.trim();

    float newPI = input.substring(3).toFloat();
    newPI = constrain(newPI, 330, 2600);
    if (newPI >= 0) {
        pi = newPI;
        manualPI = true;
        Serial.print("Manual PI set to: ");
        Serial.println(pi);
    } else {
        Serial.println("Invalid PI value.");
    }


if (input.equalsIgnoreCase("AUTO")) {
    manualPI = false;
    Serial.println("Switched to automatic PI control.");
  }
}
```

Below you can see how we enable the conduction of the triacs. The delay of 9700-pi us means that when the pi variable is big (== the error is big), the delay will be shorter so the triacs will conduct for a larger duration of the voltage half-cycle.

```
if (error >= 0 && zero_cross_detected) {
    delayMicroseconds(9700 - pi);

    for(int i=0; i<4; i++){
    digitalWrite(angleControl[i], HIGH);
    delayMicroseconds(70);
    digitalWrite(angleControl[i], LOW);
    }
    zero_cross_detected = false;
}
```

Target and endGame variables

The target variable starts from the value “10” and increases every 1.5 seconds until it reaches the value of endGame. That happens so the error is gradually increased, and the system follows a ramp approach.

The endGame value represents the desired temperature we want to achieve inside the oven.

```
int target = 10;
int endGame = 75; // SET THIS AT THE WILLING TEMPERATURE
```

```
// Increase Target
void increase() {
  if (round((endGame - target)) >= 1) {
    target += 1;
  }
}
```

```
if (target > endGame) {
  target = endGame;
}

if ((millis() - timerIncrease) >= 1500) {
    increase();
    timerIncrease = millis();
}
```

LCD display

The LCD displays the current and the desirable temperature, the target value as it increases and the error value. The code we use to achieve that is implemented inside the function monitor() which is called inside the loop every second.

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

  // LCD
LiquidCrystal_I2C lcd(0x27, 20, 4);

// Monitor LCD
void monitor() {
    lcd.clear();
    lcd.setCursor(0,0);
    lcd.print("Goal");
    lcd.setCursor(5,0);
    lcd.print(endGame);
    lcd.print(".00");
    lcd.print(" ");

    lcd.setCursor(11,0);
    lcd.print("T:");
    lcd.print(target);
    lcd.print("  ");

    lcd.setCursor(0,1);
    lcd.print("Temp");
    lcd.setCursor(5,1);
    lcd.print(read_average_temperature());

    lcd.setCursor(11,1);
    lcd.print("E:");
    lcd.setCursor(13,1);
    lcd.print(error);
    lcd.print("  ");
}
```

Potentiometer

All we need to obtain the desired temperature from the user are two variables, one to load the value that the analogRead() function returns and one to load the target temperature after we map, with the help of the map() function, the value from the analog input to 20-100°C. Also, we added a constraint just to make sure that the value will never exceed 100°C.

```
int potValue = analogRead(POTENTIOMETER);              // Read potentiometer
endGame = map(potValue, 0, 4095, 20, 100);             // Map to temp range
endGame = constrain(endGame, 20, 100);                 // Constrain just in case
```

Serial Monitor

Finally, for debugging purposes, we added some prints to the Serial Monitor, like the ones that are depicted on the LCD display, plus the PI output. These messages are printed every second when you connect your laptop to the esp32 and can function as an alternative to the LCD if something goes wrong (i.e. the display malfunctions).

```
if (millis() - timerMonitor >= 1000) {
  Serial.print("End Game : ");
  Serial.print(endGame);
  Serial.print(" | Target Temp: ");
  Serial.print(target);
  Serial.print(" | Measured Temp: ");
  Serial.print(avgTemp);
  Serial.print(" | Error: ");
  Serial.print(error);
  Serial.print(" | PI Output: ");
  Serial.println(pi);
  timerMonitor = millis();
}
```

5. Test and suggestions

To test the functionality of the PCB we check the following things by using an oscilloscope:

1. The output of the zero-volt cross circuit
2. The pulses from the angleControl pins
3. The pulses on the optocouplers on the LV side and the trigger signals on the HV side to the triac

After noticing that all the results were the expected, we supplied the PCB with the 220V and measured the output current from each triac with an amperemeter.

At first everything worked perfectly, but after some time, two triacs stopped being conducted. So, as it is reported above, we used only the two functional triacs and decreased the upper limit of the pi variable.

Finally, we think by reading this booklet you will have a brief idea of what should be done to optimize this project. Anything else you need to add, like peripherals, or some engineering you need to do to make the control to fit your specifications, well… that's up to you.

Sources – References

1. https://aristotle-university-racing-team-electric-aristurtle-2.365.altium.com/designs/59F2FFF9-056E-4E07-A582-B7CE4B444313?activeView=SCH&activeDocumentId=Sheet_Oven.SchDoc&variant=[No+Variations]&location=[1,101.46,19.05,14.08]#design

2. https://gitlab.com/aristurtle/electrical/oven

3. https://en.wikipedia.org/wiki/Thermocouple

4. https://eepower.com/resistor-guide/resistor-types/ntc-thermistor/

5. https://sound-au.com/appnotes/an005.htm

6. https://dextrel.net/dextrel-start-page/design-ideas-2/mains-zero-crossing-detector